Relatório Técnico de Execução - Pipeline ETL U-Easy

(Equipe 8)

Guilherme José
Maria Alice
Vinícius Leôncio

1. Abordagem e Análise do Projeto

A equipe do aplicativo U-Easy buscava otimizar a mobilidade urbana, mas carecia de expertise para tratar grandes volumes de dados inconsistentes. Nossa missão foi analisar a base pública de registros de viagens de táxi de Nova York (Yellow Taxi) referente a janeiro de 2024, juntamente com a tabela de mapeamento Taxi Zone Lookup.

Para solucionar esse problema, decidimos implementar as seguintes diretrizes arquiteturais e de governança:

- **Processamento 100% ELT em DuckDB:** Descartamos ferramentas visuais, planilhas ou linguagens secundárias (como Python/R) para garantir que toda a ingestão e transformação ocorresse via SQL puro.
- **Tolerância Zero a Edições Manuais e Deleções:** Decidimos não mascarar falhas. Qualquer registro inconsistente não seria apagado silenciosamente, mas desviado para uma trilha de auditoria explícita (rejected_records).
- **Governança via Servant Leadership:** A equipe operou sem "desenvolvedores heróis". Todo o código foi versionado via Git e exigiu revisões cruzadas (Pull Requests) para evitar a retenção de conhecimento em silos.
- **Idempotência:** O pipeline foi construído para ser executado múltiplas vezes de forma determinística, entregando o mesmo banco de dados estruturado em um Esquema Estrela (Star Schema) a partir dos dados brutos originais.

2. Execução das Sprints (Papéis e Entregas)

Sprint 1: Setup do ambiente e ingestão bruta

Responsável: Vinícius Leôncio

- **O que foi feito:** O projeto foi iniciado com a estruturação do ambiente local e do versionamento via Git, assegurando um histórico rastreável desde o primeiro passo. O DuckDB foi acionado para carregar nativamente os registros massivos em Parquet e o dicionário de zonas em CSV, finalizando a etapa com validações de linhas e mapeamento de tipos no terminal.

Raciocínio e Decisões Tomadas:

- **Preservação Absoluta:** A decisão mais crítica foi manter a camada bruta (raw) 100% inalterada. O raciocínio por trás disso é que uma base intocada atua como a única fonte da verdade e um pilar inegociável de governança. Caso regras de negócio mudem no futuro ou camadas superiores sejam corrompidas, a base bruta intacta garante a capacidade de reprocessar o pipeline inteiro sem perdas.
- **Isolamento de Tratamento:** Para mitigar o risco de corrupção por esquemas divergentes ou arquivos físicos, decidiu-se centralizar a carga inicialmente e delegar qualquer limpeza ou tipagem estritamente para a camada de Staging subsequente.

Alternativas Descartadas:

- Descartei a aplicação de filtros ou conversão de tipos de dados já na ingestão, evitando a perda precoce de informações que poderiam ser úteis para auditorias futuras.

Sprint 2: Profiling e Auditoria de Qualidade de Dados

Responsável: Maria Alice

O que foi feito:

- Foi criado o arquivo de varredura proffiling_queries.sql para executar uma série de

consultas analíticas profundas, visando o entendimento da base de dados bruta e a identificação de anomalias.

- Realizou-se o mapeamento estrutural completo, quantificando o volume de linhas e mapeando os tipos e nomenclaturas de cada coluna para validar os formatos disponíveis.

total_registros_raw_yellow_taxi int64
2964624 (2.96 million)

raw_yellow_taxi

VendorID	integer
tpep_pickup_datetime	timestamp
tpep_dropoff_datetime	timestamp
passenger_count	bigint
trip_distance	double
RatecodeID	bigint
store_and_fwd_flag	varchar
PULocationID	integer
DOLocationID	integer
payment_type	bigint
fare_amount	double
extra	double
mta_tax	double
tip_amount	double
tolls_amount	double
improvement_surcharge	double
total_amount	double
congestion_surcharge	double
Airport_fee	double

- Executou-se a contagem sistemática de valores nulos, isolando a necessidade de tratamentos futuros nas colunas passenger_count, ratecodeID, store_and_fwd_flag, congestion_surcharge e Airport_fee.

qtd_null_passenger_count int64
140162

qtd_null_ratecodeID int64	qtd_null_store_and_fwd_flag int64
140162	140162

qtd_null_congestion_surcharge int64	qtd_null_Airport_fee int64
140162	140162

- A cardinalidade de cada coluna foi extraída e comparada diretamente com o limite de

valores previstos no dicionário de dados oficial.

card_vendor_id int64	card_tpep_pickup_datetime int64	card_tpep_dropoff_datetime int64	card_passenger_count int64	card_trip_distance int64
3	1575706	1574780	10	4489

card_ratecodeID int64	card_store_and_fwd_flag int64	card_PULocationID int64	card_DOLocationID int64	card_payment_type int64
7	2	260	261	5

card_fare_amount int64	card_extra int64	card_mta_tax int64	card_tip_amount int64	card_tolls_amount int64
8970	48	8	4192	1127

card_improvement_surcharge int64	card_total_amount int64	card_congestion_surcharge int64	card_Airport_fee int64
5	19241	6	3

- Uma auditoria estatística (mínimo, médio e máximo) nas colunas numéricas expôs falhas críticas nos dados, como o registro de tarifas e valores monetários negativos, além de distâncias de viagens fisicamente irreais.
- Validou-se a integridade relacional cruzando os IDs de locais presentes na tabela fato bruta contra a dimensão do dicionário de localidades, atestando a inexistência de dados órfãos nesse aspecto.

Raciocínio e Decisões Tomadas:

- **Mapeamento Prévio ao Tratamento:** O processo mental adotado foi agir de forma diagnóstica antes de qualquer mutação. A decisão de contar os nulos serviu estritamente para mapear o escopo do que deveria ser corrigido e tratado na sprint seguinte, evitando falhas silenciosas na pipeline de limpeza.
- **Visão Focada no Negócio:** Ao avaliar os limites numéricos, o raciocínio não foi apenas achar erros de sintaxe, mas buscar quebras nas regras de negócio da operação de táxi. Tarifas negativas apontam diretamente para falhas de sensor ou estornos mal capturados, guiando a modelagem analítica a criar barreiras lógicas posteriormente.

Alternativas Descartadas:

- Descartei a limpeza, exclusão ou transformação imediata dos dados anômalos

identificados. Como o objetivo da etapa de profiling é apenas entender a saúde da base, aplicar filtros prematuros ocultaria o estado real da fonte e quebraria a governança dos dados.

Sprint 3: Limpeza e Camada de Staging

Responsável: Guilherme José

O que foi feito: O script create_staging_views.sql foi desenvolvido para aplicar coerções de tipo e isolar dados problemáticos em uma tabela de auditoria ativa (rejected_records). A partir disso, visões limpas (stg_yellow_taxi e stg_zone_lookup) foram geradas para alimentar a modelagem.

Raciocínio e Decisões Tomadas:

- **Auditoria vs. Exclusão:** Ao analisar os erros nos dados (ex: viagens acima de 70 mph, tarifas > $1000, viagens com tempo negativo), concluí que uma simples instrução "DELETE" destruiria a rastreabilidade. Optei por um fluxo de "Auditoria Ativa", onde as 12 regras de negócio criadas desviam a anomalia e cravam nela um rótulo do motivo da rejeição.
- **Coerção Defensiva:** A conversão implícita de tipos costuma quebrar pipelines inteiros. A decisão lógica foi envelopar 100% das colunas brutas em funções TRY_CAST. Assim, valores textuais em colunas numéricas convertem-se para nulos em vez de causarem interrupções ("crashes") no DuckDB. Além disso, todas as colunas monetárias foram estritamente convertidas para DECIMAL(18,2) ao invés de FLOAT ou DOUBLE, visando garantir precisão matemática absoluta em cálculos financeiros futuros e evitar erros de arredondamento inerentes ao ponto flutuante.
- **Proteção de Métricas de Negócio:** Nulos em colunas vitais precisavam de tratamento lógico e não poderiam apenas ser ignorados. Se o volume financeiro de taxas fosse nulo, isso quebraria as funções matemáticas SUM() nas agregações da camada analítica. Logo, decidi forçar a conversão dessas taxas nulas estritamente para 0.00 utilizando funções COALESCE. Para o volume de passageiros, nulos (ou vazios) assumiram o valor 1, refletindo o cenário operacional mais provável. Para códigos de identificação nulos ou inválidos, apliquei o mapeamento para códigos coringa definidos pelo próprio dicionário de dados oficial, convertendo os IDs de tarifas falhos (RatecodeID) para 99 e

os tipos de pagamentos inválidos para 5 (Unknown).

- **Chaves Inequívocas:** Para retirar os registros rejeitados da visão principal sem gerar exclusões acidentais de viagens parecidas, percebi que chaves compostas (Data + Valor) eram insuficientes. Adotei a pseudocoluna rowid do DuckDB para garantir um filtro fisicamente exato.

Alternativas Descartadas:

- Descartei completamente qualquer limpeza manual prévia em arquivos ou ferramentas visuais, além de descartar deleções puras que violariam o princípio de Zero-Trust Pipeline.

Sprint 4: Desenho de arquitetura dimensional

Responsável: Vinícius Leôncio

O que foi feito: O desenho conceitual e lógico do modelo analítico foi estruturado, mapeando a tabela fato central e suas respectivas dimensões, culminando na geração do Diagrama de Entidade-Relacionamento.

Raciocínio e Decisões Tomadas:

- **Otimização Colunar:** A escolha entre Star Schema e Snowflake exigiu análise da engine. Como o DuckDB possui processamento colunar vetorizado, o raciocínio foi adotar o Star Schema. A desnormalização das dimensões evita junções em cascata (joins excessivos) que sobrecarregam a CPU, acelerando a leitura analítica drasticamente.
- **Granularidade Fina:** Decidiu-se por um grão estritamente atômico: cada linha da Fato é uma única viagem. Isso preserva a máxima flexibilidade de agrupamento posterior.
- **Idempotência nas Chaves:** Para não depender de sequências auto-incrementais que falham em cargas distribuídas, idealizei o uso de algoritmos Hash MD5 gerados a partir dos atributos naturais.
- **Role-Playing Dimensions:** Para evitar a duplicação ineficiente da tabela de zonas de táxi, diagramei a dimensão de Localização como uma dimensão interpretativa

(Role-Playing), conectando-a duas vezes à fato para representar tanto origem quanto destino. Isso garante uma arquitetura limpa e de alta manutenibilidade.

Alternativas Descartadas:

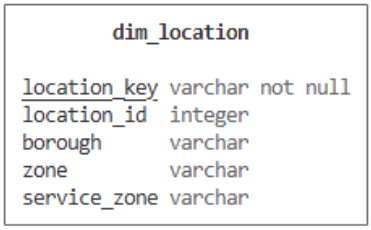
- O Snowflake Schema foi descartado devido à degradação de performance imposta por normalização excessiva em um data warehouse analítico local.

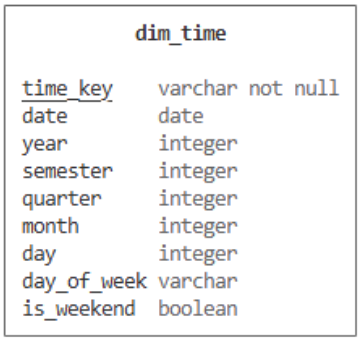
Sprint 5: Materialização Física do Modelo Dimensional

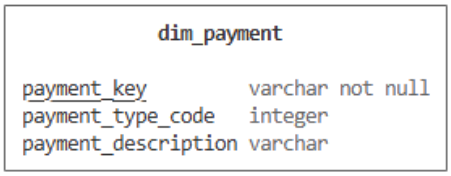
Responsável: Maria Alice

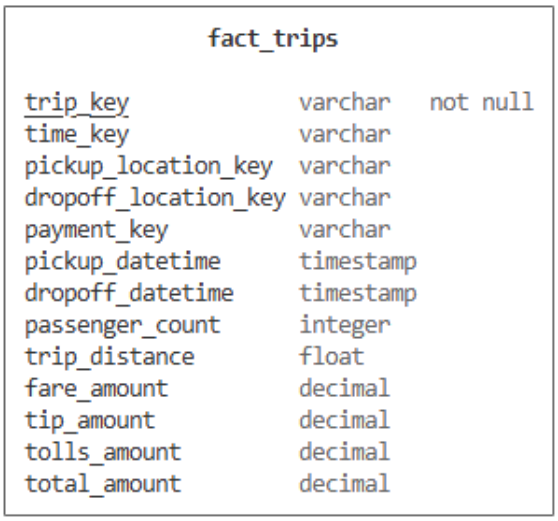
O que foi feito:

- Os arquivos create_dimensions.sql e create fact_trips.sql foram desenvolvidos para instanciar a estrutura do banco.









- As tabelas que compõem o Star Schema foram materializadas fisicamente: dimensões (dim_location, dim_time, dim_payment) e a tabela central de métricas (fact_trips).
- Adotou-se uma abordagem em duas etapas: primeiramente a definição DDL das tabelas (completas com *Primary Keys* e *Foreign Keys*) e, apenas posteriormente, a inserção dos dados já higienizados nessas estruturas.
- As chaves primárias adotaram hashes determinísticos gerados por algoritmos MD5 construídos em cima de atributos naturais consistentes. Na tabela fato, o MD5 foi aplicado sobre um conjunto agrupado de atributos validados contra duplicidade.
- Foi inserida uma linha artificial de salvaguarda (conhecida como "membro desconhecido") nas dimensões, sob o identificador 'MD5_UNKNOWN'.

Raciocínio e Decisões Tomadas:

- **Contorno de Limitações da Engine:** A decisão crítica de separar a declaração de chaves da ingestão de dados foi pautada por um entrave de software: o DuckDB não permite classificar ou alterar o vínculo de uma chave primária ou estrangeira após a tabela ter sido criada. Isso exigiu um planejamento meticuloso do esquema antes do povoamento.
- **Idempotência e Segurança Anticolisão:** O raciocínio por trás do uso de chaves Hash MD5 (em vez de *auto-increments*) foi assegurar consistência distributiva. Como chaves de tabelas fato precisam ser exatas, selecionei um conjunto de atributos naturais que foi exaustivamente testado em iterações prévias, mitigando totalmente o risco matemático

de colisões lógicas ou duplicações sistêmicas.

- **Governança Anti-Perda de Faturamento:** A adoção do 'MD5_UNKNOWN' baseia-se na necessidade de resiliência. Se uma viagem vier com atributos faltantes, o cruzamento analítico não derrubará o registro da corrida inteira. Ela é apenas redirecionada para a categoria "Desconhecida", garantindo que a volumetria total financeira da empresa permaneça inviolada.

Alternativas Descartadas:

- Descartei o uso de índices sequenciais padrão (ex: IDENTITY/SERIAL) para as dimensões, pois impossibilitariam o paralelismo de cargas e criariam disparidades de chaves caso o pipeline inteiro precisasse ser rodado em ambientes físicos diferentes.

Sprint 6: Plano de Data Quality (DQ) e Reconciliação

Responsável: Guilherme José

- **O que foi feito:** Criei um suíte profunda de testes SQL (dq_reconciliation.sql e quality_checks.sql) desenhada para avaliar matematicamente a integridade estrutural, a unicidade e o balanceamento financeiro entre as tabelas do pipeline.

Raciocínio e Decisões Tomadas:

- **Prova Matemática de Fluxo:** Meu raciocínio central foi de que apenas criar regras não garante a execução correta. Era preciso "provar matematicamente" que nada vazou. Formulei invariantes estruturais: Volumétrica (Raw = Fato + Rejeitados) e Financeira (Soma de total_amount em Raw = Soma na Fato + Soma nos Rejeitados). Se o delta numérico ou financeiro dessas equações fosse diferente de exatos zero, o pipeline estaria reprovado.

total_raw int64	total_fato int64	total_rejeitados int64	delta_linhas int64
2964624	2869161	95463	0

total_bruto decimal(38,2)	total_reconciliado decimal(38,3)	diferenca_financeira decimal(38,3)
79456384.28	79456384.280	0.000

- **Testes Lógicos Retroativos:** Após a limpeza da Sprint 3, testei os próprios filtros aplicando 12 regras de domínio (ex: viagens longas irreais > 300 milhas, falhas de GPS > 70 mph, tarifas astronômicas > $1000, ou distorções de gorjetas > 50% do valor da corrida). Escrevi essas invariantes para confirmar que nenhum registro inconsistente atingiu a Fato.
- **Autossuficiência Analítica:** Para testar a completude (garantindo zero chaves FK nulas e validando preenchimento da métrica total_amount), a unicidade de Hash (atestando zero repetições na Surrogate Key trip_key) e a integridade referencial da fato (garantindo que toda FK existe nas dimensões de tempo, localização e pagamento), optei por modularizar todos os testes via DQL (Data Query Language) padrão dentro do DuckDB.

falhas_completude int64
0

trip_key varchar	repeticoes int64

0 rows

falhas_consistencia int64
0

falhas_integridade int64
0

Alternativas Descartadas:

- Descartei amostragens de auditoria. Processar validações em lotes parciais é arriscado; estruturei o código para ler e conciliar a totalidade dos quase 3 milhões de registros simultaneamente.
- Descartei ferramentas externas (dbt tests) para preservar a simplicidade e a reprodutibilidade isolada cobrada no escopo do projeto.

Sprint 7: Camada analítica e perguntas de negócio

Responsável: Vinícius Leôncio

O que foi feito:

- O esforço foi redirecionado para a tradução da estrutura dimensional em consultas SQL (business_queries.sql) focadas nas quatro frentes (dashboards) do negócio. Abaixo estão as frentes criadas, prontas para gerar evidências visuais dos retornos analíticos:

- **Dashboard 1 (KPIs Globais de Operação e Finanças):** Criou-se uma CTE centralizada para apresentar o valor médio pago, distância média (milhas), tempo médio (minutos), velocidade média (mph), percentual de gorjeta (%) e receita por milha rodada.

avg_total_amount double	avg_trip_distance_miles double	avg_duration_minutes double	avg_speed_mph double	avg_tip_pct double	avg_fare_per_mile double
27.24	3.29	15.77	12.53	20.86	8.83

- **Dashboard 2 (Análise Espacial e Fluxos):** Usando *Role-Playing Dimensions* nas conexões de origem e destino, estruturou-se uma janela analítica (DENSE_RANK) para mapear o Top 10 rotas com maior volume de viagens da cidade de Nova York.

ranking_fluxo int64	zona_origem varchar	zona_destino varchar	volume_viagens int64
1	Upper East Side South	Upper East Side North	21645
2	Upper East Side North	Upper East Side South	19207
3	Upper East Side North	Upper East Side North	15183
4	Upper East Side South	Upper East Side South	14106
5	Midtown Center	Upper East Side South	10139
6	Lincoln Square East	Upper West Side South	8829
7	Upper East Side South	Midtown Center	8720
8	Midtown Center	Upper East Side North	8667
9	Upper West Side South	Lincoln Square East	8544
10	Upper West Side South	Upper West Side North	8308

10 rows 4 columns

- **Dashboard 3 (Análise Temporal - Picos):** Criaram-se consultas independentes para retornar o Top 5 horários de pico diários, apontar exatamente qual foi o dia individual de maior volume e extrair qual semana do mês concentrou mais fluxo via funções *DATE_TRUNC* dinâmicas.

hora_embarque int64	volume_viagens int64
18	206340
17	200237
16	184829
15	183890
19	178814

data_viagem date	dia_semana varchar	volume_viagens int64
2024-01-27	Saturday	107225

inicio_semana timestamp	volume_viagens int64
2024-01-22 00:00:00	688657

- **Dashboard 4 (Valor Médio por Dia e Faixa de Horário):** Estruturou-se uma classificação em faixas lógicas ("Manhã", "Tarde", "Noite", "Madrugada") cruzada com o dia da semana da dimensão de tempo, permitindo atestar o faturamento médio e volume de frota por momento do dia para apoiar precificação.

dia_semana varchar	faixa_horaria varchar	volume_viagens int64	valor_medio_pago double
Friday	1. Manhã (06h-11h)	91609	25.72
Friday	2. Tarde (12h-17h)	143082	27.83
Friday	3. Noite (18h-23h)	140632	27.0
Friday	4. Madrugada (00h-05h)	20990	29.17
Monday	1. Manhã (06h-11h)	91629	27.33
Monday	2. Tarde (12h-17h)	150404	28.37
Monday	3. Noite (18h-23h)	111040	29.96
Monday	4. Madrugada (00h-05h)	39942	29.99
Saturday	1. Manhã (06h-11h)	62472	24.64
Saturday	2. Tarde (12h-17h)	148474	25.09
Saturday	3. Noite (18h-23h)	144500	25.65
Saturday	4. Madrugada (00h-05h)	51817	24.42
Sunday	1. Manhã (06h-11h)	56142	25.76
Sunday	2. Tarde (12h-17h)	127090	26.96
Sunday	3. Noite (18h-23h)	84022	30.42
Sunday	4. Madrugada (00h-05h)	59441	24.22
Thursday	1. Manhã (06h-11h)	99560	25.79
Thursday	2. Tarde (12h-17h)	150826	28.55
Thursday	3. Noite (18h-23h)	151995	27.55
Thursday	4. Madrugada (00h-05h)	13717	32.12
Tuesday	1. Manhã (06h-11h)	117476	26.6
Tuesday	2. Tarde (12h-17h)	172005	27.8
Tuesday	3. Noite (18h-23h)	144984	27.18
Tuesday	4. Madrugada (00h-05h)	14629	36.11
Wednesday	1. Manhã (06h-11h)	122485	25.84
Wednesday	2. Tarde (12h-17h)	180116	27.93
Wednesday	3. Noite (18h-23h)	162296	27.33
Wednesday	4. Madrugada (00h-05h)	15786	35.94

Raciocínio e Decisões Tomadas:

- **Isolamento de Origem:** Para mitigar inconsistências, decidi estabelecer como regra de ouro que todas as queries de negócio consultariam puramente a Fato e as Dimensões. O acesso direto à base bruta foi proibido para evitar o consumo de "lixo operacional".
- **Modularidade e Legibilidade:** O raciocínio para lidar com cálculos complexos foi empregar extensivamente as CTEs (Common Table Expressions). Isso me permitiu pré-agregar os dados, realizar os cruzamentos lógicos em etapas distintas e evitar duplicação de regras lógicas, deixando o código pronto para migração para ferramentas de BI.
- **Proteção contra Fator Zero:** Sabendo que os dados operacionais contêm inconsistências residuais em cálculos de tempo, utilizei funções condicionais ativas (como o uso de NULLIF) para proteger as métricas. Calcular velocidade média dividindo pela duração não quebrará a query se a duração for zero.

- **Precisão Geográfica e Temporal:** Empreguei funções de janela (DENSE_RANK()) para classificar os corredores de viagens (ex: Aeroportos) sem omitir zonas com volumes empatados. Para a frente de precificação dinâmica, as datas foram dinamicamente extraídas e agrupadas em buckets analíticos como "manhã, tarde, noite".

Alternativas Descartadas:

- Descartei agrupar os dados por limite arbitrário (ex: LIMIT rígido sem função de ranqueamento prévia), pois cortaria dados operacionais vitais de zonas empatadas em volume de corridas.

Sprint 8: Orquestração e Execução do Pipeline

Responsável: Maria Alice

O que foi feito:

- Desenvolveu-se o script mestre run_pipeline.sql.
- Todos os comandos de execução de cada processo individual da arquitetura de dados foram agrupados dentro desse arquivo, ordenando estritamente a topologia lógica da *pipeline*.

Raciocínio e Decisões Tomadas:

- **Centralização e Reprodutibilidade:** O raciocínio central foi consolidar o ciclo de vida dos dados em uma única chamada. Dispor da capacidade de rodar todo o ecossistema com um único gatilho eleva drasticamente a praticidade da manutenção. Isso garante que qualquer membro da equipe ou sistema de orquestração externo possa recriar o ambiente analítico do zero, com garantia de consistência temporal e ordenamento correto das dependências operacionais.

Alternativas Descartadas:

- Descartei a manutenção de execuções soltas ou scripts interdependentes ativados manualmente, pois essa prática corrompe a governança e introduz altas chances de

falha humana ao tentar lembrar a ordem exata das cargas DDL, DML e validações.

3. Conclusão do Projeto

A jornada técnica deste projeto refletiu uma evolução cadenciada e metodológica. O trabalho começou com a preservação cautelosa do dado em seu estado bruto, evoluiu para a engenharia minuciosa de um esquema estrela focado na performance do banco colunar e culminou na extração limpa e refinada de inteligência de negócio.

Cada decisão, desde a imutabilidade da base inicial até a blindagem das métricas analíticas contra anomalias matemáticas, provou-se essencial. O projeto final materializou um Star Schema que blindou a operação contra perda silenciosa de dados e passou com sucesso por todos os atestados matemáticos de integridade. O repositório final entregue é uma base analítica estruturada, documentada e testada, pronta para integração com ferramentas de Business Intelligence e preparada para escalar conforme o aumento do volume de dados.